Smart Irrigation Advisor: Predicting When a Plant Needs Water with ESP32 Sensors and Machine Learning

ML 402331 — IoT Machine Learning Project · **Phase 1 Proposal** · Fall 2026/2027 · American University of Madaba

Student (individual project)	Student ID	Track	Project idea
Mustafa Juma	2410102	Hardware (instructor approval requested)	#4 Smart irrigation

1. Idea, Problem Statement and Research Question

Idea. A low-cost ESP32 node reads soil moisture, air temperature and humidity every 5 minutes. A classifier predicts whether the soil will reach the plant's critical moisture level within the next 60 minutes, so the user is warned (and later a pump can start) before the plant is stressed. **User:** anyone who looks after potted plants, a balcony garden or a small greenhouse. **Benefit:** less over- and under-watering and less wasted water, which matters in water-scarce Jordan.

Problem statement. Fixed timers and simple moisture thresholds are reactive: a threshold fires only after the soil is already dry, and it ignores that hot, dry air makes soil dry faster. I want a model that learns how temperature, humidity and the recent moisture trend affect drying, and warns early.

Research question. Using readings from my own ESP32 plant setup, can a classifier predict "soil reaches the critical level within 60 minutes" with an **F1-score of at least 0.85** on a held-out test set, and does it **beat a plain moisture-threshold rule** on the same data?

Related work.

[1] Capcha-Ochoa et al. build an ESP32 irrigation system with soil-moisture, DHT22 and light sensors and report about 43.7 % less water than fixed-schedule watering in a park test. Machine learning is used for plant-health vision, while irrigation itself follows sensor ranges.

[2] Taueatsoala et al. propose an edge-first TinyML irrigation framework (soil moisture, temperature, humidity, pH, light) and compare ensemble models, showing lightweight ML can run on low-cost hardware.

[3] Junaidi uses an ESP32 with soil-moisture, temperature and humidity inputs and a KNN model to label readings as "needs water"; the reported test is a single example input.

Gap I address: I predict ahead (60 minutes) rather than react, and I evaluate several models on held-out data with cross-validation against a threshold baseline.

2. ML Task, Target Label and Planned Features

Item	Plan
Task type	Supervised **binary classification** (primary). A secondary regression view (forecast soil moisture 60 min ahead) supports the Ridge/Lasso work in Phase 2.
Target label	needs_water_60min = 1 if soil moisture is at or below the critical level **T** at any point in the next 60 minutes, otherwise 0. T is the critical level for my plant (initially 30 % of the calibrated range, tuned in Phase 2).
Planned features	Soil moisture (% and raw ADC), air temperature, relative humidity, hour of day, moisture change over the last 15 and 30 minutes, time since last watering.
Models and evaluation	Baseline: regularised logistic regression (Phase 2). Ensemble: Random Forest and Gradient Boosting with GridSearchCV in a Pipeline (Phase 3). SVM (RBF) and DBSCAN for unusual readings (Phase 4). All compared with the threshold rule using 5-fold CV, accuracy, precision, recall, F1 and a confusion matrix.

3. Data Plan

Sensors and logging. Capacitive soil-moisture sensor and DHT22 on an ESP32, one row every 5 minutes (timestamp, raw ADC, moisture %, temperature, humidity, watering flag). Data reaches the laptop over serial into a CSV through a Python logger (MQTT or ThingSpeak as backup). **Size.** The minimum is 500 labelled rows; I target **2,000 or more** (about 7 days) covering at least 3 dry-down and re-watering cycles in different conditions (sun and shade) so both classes appear. **Labels.** Computed automatically from the logged moisture series with the look-ahead rule above, then checked against at least 30 manual checks (finger test and notes) to tune T. Train/test splits follow time order to avoid leakage.

4. Hardware Track: Bill of Materials and Tools

Component	Qty	Est. cost (USD)	Why it is needed
ESP32 DevKit (WROOM-32)	1	8.00	Reads sensors, has Wi-Fi, can run an exported model
Capacitive soil-moisture sensor v1.2	2	6.00	Main signal; capacitive type resists corrosion; one spare
DHT22 temperature/humidity sensor	1	5.00	Air conditions that change how fast soil dries
Breadboard and jumper wires	1	4.00	Solder-free prototype
10 kΩ resistor	1	0.50	DHT22 data-line pull-up
USB data cable and 5 V adapter	1	4.00	Power and serial logging
Potted plant and soil	1	5.00	Test subject for data collection
Core total		**32.50**	Optional for Phase 4 demo: relay module 2.50, mini pump 4.00, tubing 2.00 (8.50)

Prices are estimates to be confirmed with local suppliers. **Software:** Python 3, Google Colab/Jupyter, scikit-learn, pandas, NumPy, Matplotlib, Arduino IDE (C++), GitHub; joblib and emlearn/micromlgen for deployment.

5. Repository and Starter Notebook

GitHub: the repository link is in GITHUB_LINK.txt in this submission. The notebook opens in Google Colab directly from the repository (notebooks/phase1_starter_linear_regression.ipynb). The repository holds the README, notebook, bill of materials and folders for data and device code. The notebook loads scikit-learn's Diabetes dataset [4] and runs a first linear regression (test $R^2 = 0.45$, RMSE = 53.9, 5-fold CV mean $R^2 = 0.48$) with residual plots. A clearly labelled synthetic section previews my label and feature pipeline. Library: scikit-learn [5].

6. Contributions and AI Use

Student	Contribution in Phase 1
Mustafa Juma (2410102)	Individual project: idea, research question, related work, data plan, bill of materials, repository and starter notebook. Instructor approval for working alone is requested.

AI-use statement: an AI assistant (Claude) helped draft this proposal and the starter notebook. I reviewed the content and am responsible for it.

References

[1] J. M. A. Capcha-Ochoa *et al.*, "Smart irrigation system with IoT, machine learning, and solar power for efficient plant care," *Emerging Science Journal*, vol. 9, no. 3, Jun. 2025, doi: 10.28991/ESJ-2025-09-03-012.

[2] K. Taueatsoala, C. Daniels, A. J. Ramsunar, P. Bronkhorst and A. E. Ezugwu, "TinyML-enabled IoT for sustainable precision irrigation," arXiv:2601.13054, 2026.

[3] Junaidi, "The use of IoT in water utilization strategies for smart irrigation systems based on machine learning," *JURTEKSI*, vol. 11, no. 1, Dec. 2024.

[4] B. Efron, T. Hastie, I. Johnstone and R. Tibshirani, "Least angle regression," *Ann. Statist.*, vol. 32, no. 2, pp. 407–499, 2004.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.